

Task-06 Introduction to Cryptography

Chapter 1: Introduction to Cryptography

Cryptography is the science of protecting information by transforming it into a secure format that only authorized parties can access or understand. In cybersecurity, cryptography plays a foundational role in ensuring that sensitive data remains protected even if it is intercepted, stolen, or exposed during transmission or storage. At its core, cryptography is designed to solve three major security problems.

- The first is **confidentiality**, which ensures that data is readable only by intended recipients.
- The second is **integrity**, which guarantees that data has not been altered or tampered with during transmission.
- The third is **authentication**, which helps verify the identity of users, systems, or sources of information.

Cryptography is everywhere — from securing websites and emails to protecting files, passwords, and network communications. Without cryptographic mechanisms, attackers could easily read private messages, modify data, impersonate users, or inject malicious content into systems. Rather than relying on secrecy alone, cryptography uses well-defined mathematical algorithms and keys to provide security. Even if an attacker knows the algorithm being used, the data remains secure as long as the cryptographic keys are protected. This principle makes cryptography a reliable and scalable security solution for real-world systems.

Overall, cryptography is a critical pillar of cybersecurity. It enables trust in digital environments by ensuring that data remains private, accurate, and verifiable — forming the backbone of secure communication and information protection.

Chapter 2: Symmetric vs Asymmetric Encryption

Encryption techniques can broadly be divided into **symmetric encryption** and **asymmetric encryption**. Both are used to protect data, but they differ significantly in how keys are managed, how fast they operate, and where they are applied in real-world systems.

2.1 Symmetric Encryption

Symmetric encryption uses **a single shared secret key** for both encryption and decryption. This means the same key must be securely shared between the sender and the receiver before communication begins.

One of the most widely used symmetric encryption algorithms today is **Advanced Encryption Standard (AES)**. AES is designed to be fast and efficient, making it suitable for encrypting large amounts of data.

Key characteristics of symmetric encryption:

- Same key used for encryption and decryption
- Very fast and computationally efficient
- Suitable for encrypting files, disks, and network data
- Key distribution can be challenging if not managed securely

In practice, symmetric encryption is commonly used for **data-at-rest** and **bulk data encryption**, such as file encryption, database protection, and secure backups.

2.2 Asymmetric Encryption

Asymmetric encryption uses **two different but mathematically related keys**:

- a **public key**, which is shared openly
- a **private key**, which is kept secret by the owner

Data encrypted using the public key can only be decrypted using the corresponding private key. This eliminates the need to share secret keys directly.

A commonly used asymmetric encryption algorithm is **RSA**. RSA is widely used for secure key exchange, digital signatures, and authentication mechanisms.

Key characteristics of asymmetric encryption:

- Uses a public–private key pair
- More secure for key distribution
- Slower compared to symmetric encryption
- Not suitable for encrypting large files

Asymmetric encryption is mainly used for **secure key exchange**, **digital signatures**, and **identity verification**, rather than bulk data encryption.

2.3 Key Differences Between Symmetric and Asymmetric Encryption

The main differences between the two encryption approaches can be summarized as follows:

- **Key usage:**
Symmetric encryption uses one shared key, while asymmetric encryption uses two separate keys.
- **Performance:**
Symmetric encryption is faster and more efficient, whereas asymmetric encryption is slower due to complex mathematical operations.
- **Security model:**
Asymmetric encryption improves security by avoiding direct key sharing, while symmetric encryption requires secure key distribution.
- **Use cases:**
Symmetric encryption is used for encrypting large data volumes, while asymmetric encryption is used for secure communication setup and authentication.

2.4 Practical Usage in Real Systems

In real-world cybersecurity systems, **both encryption types are used together**. Asymmetric encryption is used to securely exchange a symmetric key, and once the key is shared, symmetric encryption is used to encrypt the actual data. This combination is known as **hybrid encryption** and is widely used in protocols such as HTTPS and secure messaging systems.

This chapter provides the conceptual foundation needed to understand the practical experiments performed in the following sections using **AES** and **RSA**.

Chapter 3: Practical – Symmetric Encryption using AES (OpenSSL)

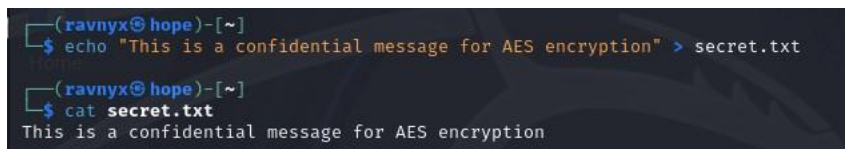
Symmetric encryption was practically implemented in this task using the **Advanced Encryption Standard (AES)** with the OpenSSL tool in Kali Linux. The objective of this experiment was to understand how data can be securely encrypted and decrypted using a single shared secret key.



```
ravnyx@hope: ~  
File Actions Edit View Help  
(ravnyx@hope)-[~]  
$ openssl version  
OpenSSL 3.2.2 4 Jun 2024 (Library: OpenSSL 3.2.2 4 Jun 2024)
```

3.1 Creating a Plaintext File

A sample text file was first created to act as sensitive data. This file contained readable information and represented data in its original, unprotected form. Verifying the plaintext before encryption helps confirm the effectiveness of the encryption process later.



```
(ravnyx@hope)-[~]  
$ echo "This is a confidential message for AES encryption" > secret.txt  
(ravnyx@hope)-[~]  
$ cat secret.txt  
This is a confidential message for AES encryption
```

Figure 3.1: Plaintext file created before AES encryption

3.2 AES Encryption using OpenSSL

The plaintext file was encrypted using **AES-256-CBC**, a strong and widely used symmetric encryption algorithm. To enhance security, **PBKDF2 (Password-Based Key Derivation Function 2)** was used along with a high iteration count. This approach makes password-based attacks significantly more difficult.

AES encryption was performed using a user-provided password, which acted as the secret key for both encryption and decryption.



```
(ravnyx@hope)-[~]  
$ openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in secret.txt -out secret_secure.enc  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:
```

Figure 3.2: AES-256 encryption performed using PBKDF2 with OpenSSL.

3.3 Verifying Encrypted Output

After encryption, the resulting file appeared as unreadable data when viewed directly. This confirmed that the encryption process successfully transformed the original plaintext into secure ciphertext that cannot be interpreted without the correct key.

```
(ravnyx@hope)-[~]
$ ls
Desktop      Templates      md5_hash.txt      rsa_encrypted.bin  signature.bin
Documents    Videos        md5_letmein.txt   rsa_message.txt    signed_message.hash
Downloads    bcrypt_hash.txt passwords.txt       secret.enc          signed_message.txt
Music         decrypted_secure.txt private.pem        secret.txt          task3_network_capture.pcapng
Pictures     integrity.hash  public.pem         secret_secure.enc
Public       integrity.txt   rsa_decrypted.txt  sha256_hash.txt
```

Figure 3.3: The encrypted output file appears unreadable, confirming that the plaintext has been securely transformed into ciphertext.

3.4 AES Decryption

The encrypted file was then decrypted using the same password and cryptographic parameters. Successful decryption restored the original content exactly, demonstrating that AES encryption preserves data integrity when the correct key is used.

```
(ravnyx@hope)-[~]
$ openssl enc -aes-256-cbc -d -pbkdf2 -iter 100000 -in secret_secure.enc -out decrypted_secure.txt
enter AES-256-CBC decryption password:
```

Figure 3.4: This screenshot shows the decryption process using the correct password, restoring the original content successfully.

```
(ravnyx@hope)-[~]
$ openssl enc -aes-256-cbc -d -pbkdf2 -iter 100000 -in secret_secure.enc -out decrypted_secure.txt
enter AES-256-CBC decryption password:

(ravnyx@hope)-[~]
$ cat decrypted_secure.txt
This is a confidential message for AES encryption
```

Figure 3.5: Successful decryption verifies the correctness of the encryption process and confirms data confidentiality and integrity.

3.5 Why AES Is Fast and Widely Used

AES is designed for efficiency and speed, making it suitable for encrypting large amounts of data. It is commonly used for:

- File and disk encryption
- Secure data storage
- VPNs and network encryption

- Secure backups

Due to its performance and strong security guarantees, AES is considered the industry standard for symmetric encryption.

This practical experiment demonstrates how symmetric encryption protects data confidentiality and highlights the importance of secure key management when using password-based encryption.

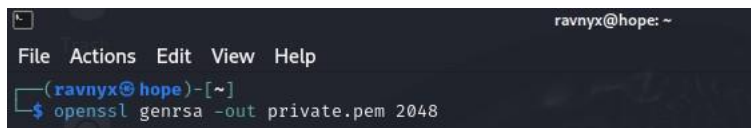
Chapter 4: Asymmetric Encryption using RSA

Asymmetric encryption was implemented in this task using the **RSA algorithm** with OpenSSL. Unlike symmetric encryption, RSA uses a **pair of keys**—a public key and a private key—to secure data. This experiment demonstrates how RSA enables secure communication without sharing a secret key.

4.1 RSA Key Pair Generation

The first step involved generating an RSA private key with a key length of 2048 bits. This private key is confidential and must be protected, as it is used to decrypt data and create digital signatures.

From the private key, a corresponding public key was generated. The public key can be shared openly and is used to encrypt data intended for the private key holder.

A terminal window with a dark background. The prompt is 'ravnyx@hope: ~'. The command 'openssl genrsa -out private.pem 2048' is entered and executed. The output is not visible, indicating successful execution.

```
ravnyx@hope: ~  
File Actions Edit View Help  
(ravnyx@hope)-[~]  
$ openssl genrsa -out private.pem 2048
```

Figure 4.1: RSA private key generation using OpenSSL

A terminal window with a dark background. The prompt is 'ravnyx@hope: ~'. The command 'openssl rsa -in private.pem -pubout -out public.pem' is entered and executed. The output 'writing RSA key' is visible. To the right of the terminal window, the text 'Public key derived' is displayed.

```
(ravnyx@hope)-[~]  
$ openssl rsa -in private.pem -pubout -out public.pem  
writing RSA key
```

Public key derived

Figure 4.1.1: The public key is extracted from the private key and is used for encrypting data securely.

4.2 Encrypting Data Using the Public Key

A sample text file was created and encrypted using the RSA public key. Once encrypted, the file content became unreadable, demonstrating how public-key encryption protects data confidentiality even when the encrypted data is exposed.

This step highlights an important property of asymmetric encryption: **anyone can encrypt data using the public key, but only the private key owner can decrypt it.**

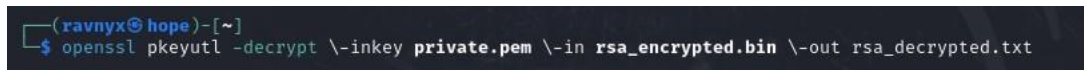
A terminal window with a dark background. The prompt is 'ravnyx@hope: ~'. The first command is 'echo "This message is protected using RSA encryption" > rsa_message.txt'. The second command is 'cat rsa_message.txt', which outputs 'This message is protected using RSA encryption'.

```
(ravnyx@hope)-[~]  
$ echo "This message is protected using RSA encryption" > rsa_message.txt  
(ravnyx@hope)-[~]  
$ cat rsa_message.txt  
This message is protected using RSA encryption
```

Figure 4.2: Plaintext message before RSA encryption, this screenshot shows the readable message that will be encrypted using asymmetric encryption.

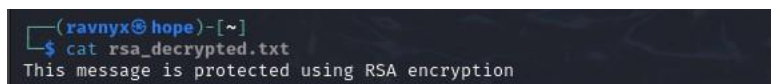
4.3 Decrypting Data Using the Private Key

The encrypted file was decrypted using the RSA private key. Successful decryption restored the original plaintext, confirming that the correct key pair was used and that the encryption process worked as expected.



```
(ravnyx@hope)-[~]  
$ openssl pkeyutl -decrypt -inkey private.pem -in rsa_encrypted.bin -out rsa_decrypted.txt
```

Figure 4.3: RSA decryption using private key, this step demonstrates that only the corresponding private key can successfully decrypt the encrypted data.



```
(ravnyx@hope)-[~]  
$ cat rsa_decrypted.txt  
This message is protected using RSA encryption
```

Figure 4.3.1: Original message recovered after RSA decryption; Successful decryption confirms the correct functioning of the RSA key pair.

4.4 Practical Limitations of RSA

While RSA provides strong security for key exchange and authentication, it is computationally expensive and not suitable for encrypting large files. For this reason, RSA is typically used to securely exchange symmetric keys rather than encrypt bulk data.

In real-world systems, RSA and AES are combined in a **hybrid encryption model**, where RSA secures the key exchange and AES handles data encryption.

This practical exercise demonstrates the role of RSA in secure communication and highlights why asymmetric encryption is a critical component of modern cryptographic systems.

Chapter 5: Hashing and Integrity Verification

Hashing is a cryptographic technique used to **verify data integrity**, ensuring that information has not been altered or tampered with. Unlike encryption, hashing does not aim to hide data; instead, it creates a unique digital fingerprint of the data.

5.1 What Is Hashing

A hash function takes an input of any size and produces a **fixed-length output**, known as a hash value or digest. Cryptographic hash functions are designed so that:

- The same input always produces the same hash
- Even a small change in input results in a completely different hash
- It is computationally infeasible to reverse the hash to obtain the original data

These properties make hashing ideal for integrity verification.



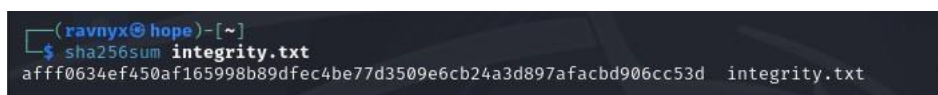
```
ravnyx@hope: ~  
File Actions Edit View Help  
(ravnyx@hope)-[~]  
$ echo "This file is used for integrity verification" > integrity.txt  
(ravnyx@hope)-[~]  
$ cat integrity.txt  
This file is used for integrity verification
```

Figure 5.1: Original file created for integrity verification; this file serves as the baseline data whose integrity will be verified using hashing.

5.2 SHA-256 Hash Generation

In this task, the **SHA-256** algorithm was used to generate hash values for files. SHA-256 is part of the SHA-2 family and is widely used due to its strong resistance to collisions and preimage attacks.

A hash was generated for a sample file, creating a reference value that represents the original file content. This hash serves as a baseline for integrity checks.



```
(ravnyx@hope)-[~]  
$ sha256sum integrity.txt  
affff0634ef450af165998b89dfec4be77d3509e6cb24a3d897afacbd906cc53d integrity.txt
```

Figure 5.2: SHA-256 hash generated for original file, the generated hash represents a unique fingerprint of the file's content.

```

(ravnyx@hope)-[~]
$ sha256sum integrity.txt > integrity.hash

(ravnyx@hope)-[~]
$ cat integrity.hash
afff0634ef450af165998b89dfec4be77d3509e6cb24a3d897afacbd906cc53d integrity.txt

```

Figure 5.2.1: Hash value stored for integrity verification, saving the hash allows future comparison to detect any unauthorized changes.

5.3 Integrity Verification Using Hashes

To verify file integrity, the stored hash value was compared with a newly generated hash of the same file. When the file remained unchanged, the hash values matched, confirming data integrity.

After modifying the file, the integrity verification failed, demonstrating how hashing can reliably detect unauthorized changes. This shows that hashing is highly sensitive to even minor data modifications.

```

(ravnyx@hope)-[~]
$ sha256sum -c integrity.hash
integrity.txt: OK

```

Figure 5.3: Successful integrity verification using SHA-256, this confirms that the file has not been modified since the hash was generated.

```

(ravnyx@hope)-[~]
$ echo "Malicious change" >> integrity.txt

(ravnyx@hope)-[~]
$ sha256sum -c integrity.hash
integrity.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match

(ravnyx@hope)-[~]
$

```

Figure 5.3.1: Integrity verification failed after file modification, the failed check demonstrates how hashing detects even minor file changes.

5.4 Difference Between Hashing and Encryption

Although both hashing and encryption use cryptographic algorithms, they serve different purposes:

- **Hashing** is a one-way process used to verify integrity
- **Encryption** is a reversible process used to protect confidentiality

Hashing does not use keys and cannot be reversed, whereas encryption relies on keys and allows data recovery through decryption.

5.5 Importance of Hashing in Cybersecurity

Hashing is widely used in cybersecurity for:

- File integrity verification
- Secure password storage
- Digital signatures
- Malware detection and identification

By ensuring that data remains unchanged, hashing plays a critical role in maintaining trust and security in digital systems.

Chapter 6: Digital Signatures (Conceptual Understanding and Demonstration)

Digital signatures are a cryptographic mechanism used to ensure **data integrity**, **authenticity**, and **non-repudiation**. Unlike encryption, which focuses on keeping data secret, digital signatures focus on **proving who created or approved the data and whether it has been altered**.

6.1 What Are Digital Signatures

A digital signature is created by generating a cryptographic hash of the data and then encrypting that hash using the sender's **private key**. The resulting output is known as the digital signature.

Anyone with access to the sender's **public key** can verify the signature, confirming that:

- The data was signed by the legitimate owner of the private key
- The data has not been modified since it was signed

A terminal window with a dark background and light-colored text. The prompt is 'ravnix@hope: ~'. The user enters 'echo "This document is digitally signed using RSA" > signed_message.txt'. The prompt changes to '(ravnix@hope)-[~]'. The user then enters 'cat signed_message.txt'. The output is 'This document is digitally signed using RSA'.

Figure 6.1: File prepared for digital signature creation; this file is used as the input for generating a digital signature.

6.2 How RSA and Hashing Enable Digital Signatures

Digital signatures rely on a combination of:

- **Hashing algorithms** (such as SHA-256) to create a fixed-length digest of the data
- **Asymmetric encryption** (such as RSA) to securely sign the hash

During verification, the recipient recalculates the hash of the received data and compares it with the decrypted signature. If both hashes match, the signature is considered valid.

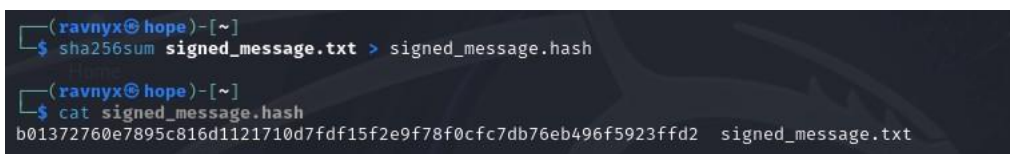
A terminal window with a dark background and light-colored text. The prompt is 'ravnix@hope)-[~]'. The user enters 'sha256sum signed_message.txt > signed_message.hash'. The prompt changes to '(ravnix@hope)-[~]'. The user then enters 'cat signed_message.hash'. The output is 'b01372760e7895c816d1121710d7fdf15f2e9f78f0cfc7db76eb496f5923fffd2 signed_message.txt'.

Figure 6.2: SHA-256 hash generated for digital signing, hashing ensures that the signature represents the exact content of the file.

```
(ravnyx@hope)-[~]  
$ openssl dgst -sha256 -sign private.pem -out signature.bin signed_message.txt
```

Figure 6.2.1: Digital signature generated using RSA private key, the hash of the file is signed using the private key, creating a digital signature.

6.3 Signature Verification and Tamper Detection

If the signed data is modified after the signature is created, the recalculated hash will differ from the original one. As a result, signature verification will fail. This makes digital signatures highly effective at detecting tampering and unauthorized modifications.

```
(ravnyx@hope)-[~]  
$ openssl dgst -sha256 -sign private.pem -out signature.bin signed_message.txt  
  
(ravnyx@hope)-[~]  
$ openssl dgst -sha256 -verify public.pem -signature signature.bin signed_message.txt  
Verified OK
```

Figure 6.3: Digital signature verification successful, verification confirms the authenticity and integrity of the signed file.

```
(ravnyx@hope)-[~]  
$ echo "Unauthorized modification" >> signed_message.txt  
  
(ravnyx@hope)-[~]  
$ openssl dgst -sha256 -verify public.pem -signature signature.bin signed_message.txt  
Verification failure  
40775E0A577F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:426:  
40775E0A577F0000:error:1C880004:Provider routines:rsa_verify:RSA lib:../providers/implementations/signature/rsa_sig.c:785:  
  
(ravnyx@hope)-[~]  
$
```

Figure 6.3.1: Digital signature verification failed after modification; this demonstrates how digital signatures detect tampering and ensure trust.

6.4 Importance of Digital Signatures in Cybersecurity

Digital signatures are widely used in:

- Secure software distribution
- Email security
- Authentication mechanisms
- Secure communications and transactions

- They play a critical role in building trust in digital systems by ensuring that data originates from a verified source and remains unchanged.

This experiment demonstrated how digital signatures combine hashing and asymmetric cryptography to provide strong guarantees of integrity and authenticity, making them a foundational component of modern cybersecurity systems.

Chapter 7: Comparison of Cryptographic Algorithms

Different cryptographic techniques serve different security purposes. The table below compares **AES**, **RSA**, and **Hashing** based on their functionality, performance, and real-world usage.

Table 7.1: Comparison of AES, RSA, and Hashing

Feature	AES (Symmetric Encryption)	RSA (Asymmetric Encryption)	Hashing (SHA-256)
Type	Symmetric encryption	Asymmetric encryption	One-way function
Keys Used	Single shared secret key	Public key & private key	No keys
Reversibility	Reversible (can decrypt)	Reversible (with private key)	Irreversible
Primary Purpose	Data confidentiality	Secure key exchange & authentication	Data integrity
Speed	Very fast	Slower	Very fast
Suitable for Large Data	Yes	No	Not applicable
Key Distribution	Difficult	Secure (public key sharing)	Not required
Output	Encrypted ciphertext	Encrypted ciphertext	Fixed-length hash
Example Algorithms	AES-128, AES-256	RSA-2048, RSA-4096	SHA-256, SHA-512
Common Use Cases	File encryption, VPNs, disk encryption	HTTPS, digital signatures, SSH	Password storage, file integrity

Feature	AES (Symmetric Encryption)	RSA (Asymmetric Encryption)	Hashing (SHA-256)
Detects Data Modification	No	No	Yes
Provides Authentication	No	Yes (with signatures)	No

Key Takeaway

Each cryptographic technique plays a **distinct role** in cybersecurity. AES is used for efficient data encryption, RSA is used for secure key exchange and authentication, and hashing is used to verify data integrity. Modern systems combine all three techniques to provide comprehensive security.

Chapter 8: Real-World Applications of Cryptography

Cryptography is not limited to theoretical concepts or laboratory experiments. It is deeply integrated into everyday digital systems and plays a crucial role in securing modern technology. Below are some of the most common real-world applications of cryptography.

1. Secure Web Communication (HTTPS)

HTTPS uses cryptographic protocols such as TLS to encrypt data exchanged between users and websites. This ensures confidentiality and protects sensitive information like login credentials and payment details from interception.

2. Virtual Private Networks (VPNs)

VPNs use encryption to create secure tunnels between devices and networks. This protects data from eavesdropping, especially when using public or untrusted networks.

3. Password Storage and Authentication

Passwords are not stored in plaintext. Instead, cryptographic hash functions are used to store password hashes, reducing the risk of credential exposure even if databases are compromised.

4. Digital Certificates and Identity Verification

Digital certificates use asymmetric cryptography to verify the identity of websites, servers, and users. They help establish trust between communicating parties in online environments.

5. Secure File and Disk Encryption

Encryption algorithms like AES are used to protect files, folders, and entire disks. This ensures that sensitive data remains inaccessible even if storage devices are lost or stolen.

6. Secure Email Communication

Cryptographic techniques such as encryption and digital signatures are used to protect email content and verify the sender's identity, preventing unauthorized access and impersonation.

7. Software Distribution and Updates

Digital signatures are used to verify the authenticity and integrity of software updates. This prevents attackers from distributing malicious or tampered software.

8. Cloud Data Security

Cloud service providers use encryption to protect stored data and data in transit. This ensures confidentiality and integrity across shared and distributed environments.

9. Online Banking and Financial Transactions

Cryptography secures financial data, transactions, and communications in online banking systems, protecting users from fraud and unauthorized access.

10. Secure Messaging Applications

End-to-end encryption is used in messaging applications to ensure that only intended recipients can read messages, even if communication channels are compromised.

Summary of Applications

These applications demonstrate how cryptography enables secure communication, protects sensitive data, and builds trust in digital systems. Without cryptographic mechanisms, modern digital services would be highly vulnerable to attacks.

Chapter 9: Final Observations and Conclusion

This task provided a clear understanding of how cryptography is used to secure digital data through both theory and practical implementation. By performing encryption, hashing, and digital signature experiments using OpenSSL, the core concepts of confidentiality, integrity, and authentication were explored.

Overall, the task demonstrated how different cryptographic techniques work together in real-world systems and reinforced the importance of cryptography as a fundamental component of cybersecurity.